

PERTEMUAN 7

LINKED LIST

1. DESKRIPSI MODUL

Linked List merupakan salah satu struktur data linear dinamis yang digunakan untuk menyimpan sekumpulan data dalam bentuk rangkaian node yang saling terhubung melalui pointer (referensi). Berbeda dengan Array yang menyimpan data pada lokasi memori yang berurutan (contiguous memory), Linked List menyimpan data pada lokasi memori yang dapat tersebar, kemudian dihubungkan menggunakan alamat referensi.

Struktur data Linked List dikembangkan untuk mengatasi keterbatasan Array, khususnya dalam operasi penambahan dan penghapusan data. Pada Array, proses penyisipan dan penghapusan data sering memerlukan pergeseran elemen sehingga kurang efisien untuk data berukuran besar. Linked List memungkinkan operasi tersebut dilakukan dengan lebih fleksibel karena hanya memerlukan perubahan referensi antar node.

Dalam dunia industri, Linked List digunakan pada implementasi playlist musik, navigasi browser, manajemen memori sistem operasi, editor teks, aplikasi undo-redo, sistem reservasi, hingga berbagai struktur data lanjutan seperti Stack, Queue, Hash Table, dan Graph.

Pada modul ini mahasiswa akan mempelajari konsep dasar Linked List, struktur node, jenis-jenis Linked List, operasi dasar, implementasi menggunakan Python, analisis kompleksitas algoritma, serta penerapan Linked List dalam berbagai studi kasus nyata.

2. TUJUAN PRAKTIKUM

Setelah mengikuti praktikum ini mahasiswa diharapkan mampu:

1. Memahami konsep dasar Linked List.
2. Menjelaskan perbedaan Array dan Linked List.
3. Memahami struktur node pada Linked List.
4. Mengimplementasikan Single Linked List menggunakan Python.
5. Mengimplementasikan operasi insert, delete, search, dan traversal.
6. Memahami konsep Double Linked List dan Circular Linked List.
7. Menganalisis kompleksitas operasi pada Linked List.
8. Mengembangkan aplikasi sederhana berbasis Linked List.
9. Mengevaluasi kelebihan dan kekurangan Linked List dibanding Array.

3. CAPAIAN PEMBELAJARAN MODUL (CPM)

Mahasiswa mampu:

CPM 1

Menjelaskan konsep dan karakteristik Linked List.

CPM 2

Mengimplementasikan struktur node dan Linked List menggunakan Python.

CPM 3

Mengimplementasikan operasi dasar Linked List.

CPM 4

Menganalisis efisiensi algoritma pada Linked List.

CPM 5

Menerapkan Linked List dalam penyelesaian masalah dunia nyata.

CPM 6

Membandingkan penggunaan Array dan Linked List pada berbagai kasus.

4. DASAR-DASAR TEORI

4.1 Pengertian Linked List

Linked List adalah struktur data linear yang terdiri dari sejumlah node yang saling terhubung.

Setiap node memiliki dua komponen utama:

1. Data
2. Pointer (referensi ke node berikutnya)

Visualisasi:

HEAD

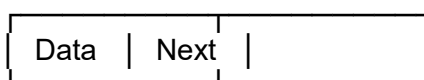
↓

[10 | •] → [20 | •] → [30 | •] → **NULL**

4.2 Struktur Node

Node merupakan elemen dasar pembentuk Linked List.

Visualisasi Node:



Contoh:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

4.3 Mengapa Linked List Dibutuhkan?

Array memiliki beberapa keterbatasan:

Kelemahan Array

- Ukuran relatif tetap
- Insert di tengah membutuhkan pergeseran data
- Delete di tengah membutuhkan pergeseran data

Contoh:

[10] [20] [30] [40]

Insert 25:

[10] [20] [25] [30] [40]

Elemen harus digeser.

Keunggulan Linked List

- Ukuran dinamis
- Insert lebih fleksibel
- Delete lebih fleksibel
- Tidak memerlukan memori berurutan

4.4 Jenis-Jenis Linked List

A. Singly Linked List

Node hanya memiliki satu pointer.

$A \rightarrow B \rightarrow C \rightarrow \text{NULL}$

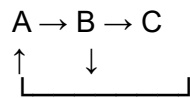
B. Doubly Linked List

Node memiliki dua pointer.

$\text{NULL} \leftarrow A \leftrightarrow B \leftrightarrow C \rightarrow \text{NULL}$

C. Circular Linked List

Node terakhir menunjuk kembali ke node pertama.



4.5 Operasi Dasar Linked List

A. Traversal

Membaca seluruh node.

HEAD → Node1 → Node2 → Node3

Kompleksitas:

$$O(n)$$

B. Insertion

Menambahkan node baru.

Jenis insertion:

- Insert awal
- Insert tengah
- Insert akhir

Insert Awal

Sebelum

10 → 20 → 30

Sesudah

5 → 10 → 20 → 30

Insert Akhir

10 → 20 → 30 → 40

C. Deletion

Menghapus node tertentu.

10 → 20 → 30

hapus 20

10 → 30

D. Searching

Mencari data tertentu.

Kompleksitas:

$O(n)$

4.6 Implementasi Linked List Menggunakan Python

Membuat Node

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

Membuat Linked List

```
class LinkedList:
    def __init__(self):
        self.head = None
```

4.7 Traversal Linked List

```
def tampilkan(self):
    current = self.head

    while current:
        print(current.data)
        current = current.next
```

4.8 Analisis Kompleksitas

Operasi	Kompleksitas
Akses Elemen	$O(n)$
Traversal	$O(n)$
Search	$O(n)$
Insert Awal	$O(1)$
Insert Akhir	$O(n)$
Delete Awal	$O(1)$

4.9 Perbandingan Array dan Linked List

Aspek	Array	Linked List
Memori	Berurutan	Tidak Berurutan
Akses Data	$O(1)$	$O(n)$
Insert	Mahal	Efisien
Delete	Mahal	Efisien
Ukuran	Tetap	Dinamis

4.10 Implementasi Linked List dalam Dunia Nyata

Playlist Musik

Lagu berikutnya terhubung ke lagu selanjutnya.

Browser History

Halaman web tersimpan dalam urutan kunjungan.

Navigasi GPS

Rute direpresentasikan sebagai rangkaian lokasi.

Editor Dokumen

Undo dan Redo.

Sistem Reservasi

Data pelanggan tersusun secara dinamis.

Manajemen Memori Sistem Operasi

Linked List digunakan untuk pengelolaan blok memori.

5. ALAT DAN PERANGKAT

Perangkat Keras

- Laptop / PC
- RAM minimal 4 GB
- Penyimpanan minimal 10 GB

Perangkat Lunak

- Python 3.x
- Visual Studio Code
- PyCharm
- Google Colab

- Browser

6. PROSEDUR PRAKTIKUM

Praktikum 1

Membuat Node

Langkah Kerja

1. Buat class Node.
2. Simpan data dalam node.
3. Tampilkan isi node.

Program

```
class Node:
    def __init__(self, data):
        self.data = data

node1 = Node(10)

print(node1.data)
```

Praktikum 2

Membuat Linked List Sederhana

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

node1 = Node(10)
node2 = Node(20)

node1.next = node2

print(node1.next.data)
```

Praktikum 3

Traversal Linked List

Buat Linked List berisi:

10 → 20 → 30 → 40

Tampilkan seluruh data.

Praktikum 4

Insert Node

Tambahkan node baru di awal dan akhir Linked List.

Praktikum 5

Delete Node

Hapus node tertentu dari Linked List.

Praktikum 6

Search Data

Cari nilai tertentu dalam Linked List.

Praktikum 7

Doubly Linked List

Implementasikan node dengan pointer:

- prev
- next

Praktikum 8

Circular Linked List

Buat node terakhir menunjuk kembali ke node pertama.

7. STUDI KASUS

Studi Kasus 1

Playlist Musik Digital

Buat playlist yang memungkinkan penambahan lagu baru secara dinamis.

Studi Kasus 2

Browser History

Implementasikan riwayat kunjungan halaman web.

Studi Kasus 3

Daftar Anggota Koperasi

Data anggota dapat ditambah dan dihapus kapan saja.

Studi Kasus 4

Sistem Reservasi Hotel

Data reservasi tersimpan secara dinamis.

Studi Kasus 5

Sistem Antrian Rumah Sakit

Pasien baru ditambahkan secara dinamis.

Studi Kasus 6

Undo dan Redo Editor Teks

Gunakan Doubly Linked List.

8. TUGAS PRAKTIKUM

Level Mudah

Tugas 1

Buat Node sederhana dan tampilkan datanya.

Tugas 2

Buat Linked List berisi 5 angka.

Tugas 3

Lakukan traversal seluruh node.

Tugas 4

Tambahkan node di awal Linked List.

Level Menengah

Tugas 5

Tambahkan node di akhir Linked List.

Tugas 6

Hapus node tertentu.

Tugas 7

Cari data tertentu pada Linked List.

Tugas 8

Hitung jumlah node dalam Linked List.

Level Sulit**Tugas 9**

Implementasikan Doubly Linked List.

Tugas 10

Implementasikan Circular Linked List.

Tugas 11

Buat sistem playlist musik menggunakan Linked List.

Tugas 12

Buat browser history sederhana menggunakan Doubly Linked List.

Tugas 13

Analisis kompleksitas seluruh operasi pada tugas 9–12.

9. DISKUSI

1. Apa yang dimaksud dengan Linked List?
2. Mengapa Linked List disebut struktur data dinamis?
3. Apa perbedaan Array dan Linked List?
4. Apa fungsi pointer pada Linked List?
5. Apa perbedaan Singly dan Doubly Linked List?
6. Apa keuntungan Circular Linked List?
7. Mengapa akses data pada Linked List lebih lambat dibanding Array?
8. Pada kondisi apa Linked List lebih baik digunakan dibanding Array?
9. Sebutkan penerapan Linked List dalam sistem komputer modern.
10. Mengapa Linked List menjadi dasar banyak struktur data lain?

10. FORMAT LAPORAN PRAKTIKUM

1. Halaman Judul
2. Tujuan Praktikum
3. Dasar Teori
4. Algoritma/Pseudocode
5. Program Python
6. Hasil Eksekusi Program
7. Screenshot Program
8. Analisis Hasil
9. Jawaban Diskusi
10. Kesimpulan
11. Daftar Pustaka

11. RUBRIK PENILAIAN

Komponen Penilaian	Bobot
Kehadiran	10%
Persiapan Praktikum	10%
Implementasi Program	25%
Ketepatan Output	20%
Analisis dan Pembahasan	15%
Laporan Praktikum	10%
Keaktifan Diskusi	10%
Total	100%

Kriteria Penilaian

Nilai	Kategori	Deskripsi
85–100	Sangat Baik	Implementasi lengkap, analisis mendalam, memahami seluruh operasi Linked List
70–84	Baik	Program berjalan baik dan analisis cukup lengkap
55–69	Cukup	Program berjalan sebagian, masih terdapat kesalahan logika
<55	Kurang	Program tidak berjalan atau laporan tidak lengkap